Modula-3 / C++ Interoperability: Design Notes from a Toy Binding

Mika Nyström

May 8, 2026

Contents

1 Introduction

Modula-3 (M3) can call C functions via `EXTERNAL` declarations, but C++ libraries have no C calling convention. This document describes a three-layer binding architecture demonstrated with a toy C++ class hierarchy, and identifies the design decisions that generalize to real libraries like Open3D, Eigen, or any C++ API.

Source code:

- https://github.com/mikanystrom/intel-async/tree/al-ga-surface-analysis/async-toolkit/m3utils/m3utils/cxxbridge

2 Architecture

The binding consists of three layers:

Layer	Language	Role
C++ library	C++	The real implementation (`shapes.hpp`)
C bridge	C++ (`extern "C"`)	Flat C API over C++ objects (`shapes_c.h`)
M3 binding	Modula-3	Idiomatic M3 types and GC integration

The C bridge is compiled as a static library (`libtoyshapes.a`) and linked into the M3 program along with `-lc++` for the C++ runtime.

3 Object Lifetime

3.1 The problem

C++ uses RAII: objects are destroyed when they go out of scope or when `delete` is called. M3 uses garbage collection: objects are destroyed at an unpredictable time after they become unreachable. When M3 holds a handle to a C++ object, who calls `delete`?

3.2 Solution: weak-reference cleaners

Each M3 wrapper object registers a **weak-reference cleaner** at construction time via `WeakRef.FromRef`. When the M3 GC collects the wrapper, the cleaner calls the C destructor:

```
REVEAL Shape = BRANDED REF RECORD
  handle : ADDRESS; (* C++ object pointer *)
  owned : BOOLEAN; (* TRUE = we should free on GC *)
END;

PROCEDURE ShapeCleaner(READONLY w: WeakRef.T; r: REFANY) =
  VAR s := NARROW(r, Shape);
  BEGIN
    IF s.owned AND s.handle # NIL THEN
      ToyShapesC.ShapeFree(s.handle);
      s.handle := NIL;
    END;
  END ShapeCleaner;
```

3.3 Ownership transfer

When a Shape is added to a ShapeList, the C++ side takes ownership of the handle. The M3 side sets `owned := FALSE` to suppress the destructor:

```
PROCEDURE Add(sl: ShapeList; s: Shape) =
  BEGIN
    ToyShapesC.ShapeListAdd(sl.handle, s.handle);
    s.owned := FALSE; (* C++ owns it now *)
    (* Keep M3 reference so GC doesn't collect the wrapper *)
    sl.members[sl.nMembers] := s;
    INC(sl.nMembers);
  END Add;
```

This is the M3 equivalent of `std::unique_ptr::release()`.

4 Type Fidelity

The C++ class hierarchy (`Shape`, `Circle`, `Rectangle`) is represented on the M3 side as a **single opaque type** `Shape`. Virtual dispatch happens entirely on the C++ side—the M3 code calls `ToyShapesC.ShapeArea(s.handle)`, which the C bridge casts to `Shape*` and calls the virtual method.

An alternative would be to mirror the hierarchy in M3:

```
TYPE Shape <: REFANY;
     Circle <: Shape;
     Rectangle <: Shape;
```

This would give M3 code compile-time type safety (e.g., calling `Radius()` on a non-Circle would be a type error). The cost is maintaining a parallel type hierarchy and doing a `TYPECASE` or `NARROW` at the bridge layer.

For this prototype, the single-type approach was chosen for simplicity. Real bindings should consider which approach better serves the users.

5 String Marshaling

C++ `std::string` cannot cross the C boundary directly. The C bridge returns a `malloc`'d `char*`:

```
char *toy_shape_name(toy_shape_t s) {
    std::string n = static_cast<Shape*>(s)->name();
    char *r = (char*)malloc(n.size() + 1);
    if (r) strcpy(r, n.c_str());
    return r;
}
```

The M3 side converts to `TEXT` and frees the C buffer:

```
PROCEDURE Name(s: Shape): TEXT =
  VAR cstr := ToyShapesC.ShapeName(s.handle);
  BEGIN
    t := M3toC.CopyStoT(cstr);
    Cstdlib.free(cstr);
    RETURN t;
  END Name;
```

`M3toC.CopyStoT` copies the bytes into a GC-managed `TEXT`, so the `free()` is safe immediately after.

6 Callbacks

6.1 The problem

C++ `std::function` captures state (closures), but C function pointers do not. The standard C pattern is a function pointer plus a `void*` context:

```
typedef void (*toy_visitor_fn)(toy_shape_t s, size_t i, void *ctx);
void toy_shapelist_foreach(toy_shapelist_t sl,
                           toy_visitor_fn fn, void *ctx);
```

On the M3 side, we want to pass an object with a method (a closure):

```
TYPE VisitorClosure = OBJECT METHODS
  visit(s: Shape; index: CARDINAL);
END;
```

6.2 Solution: trampoline + LOOPHOLE

The M3 side passes a **trampoline** procedure as the function pointer, and the closure object's address as the context:

```
PROCEDURE ForEach(sl: ShapeList; v: VisitorClosure) =
  BEGIN
    ToyShapesC.ShapeListForEach(
      sl.handle, VisitorTrampoline, LOOPHOLE(v, ADDRESS));
  END ForEach;

PROCEDURE VisitorTrampoline(h: Handle; index: CARDINAL;
                            ctx: ADDRESS) =
  VAR v := LOOPHOLE(ctx, VisitorClosure);
      s := NEW(Shape, handle := h, owned := FALSE);
  BEGIN
    v.visit(s, index);
  END VisitorTrampoline;
```

The trampoline has a fixed address (it is a top-level procedure, not a closure), so it is safe to pass as a C function pointer. The `LOOPHOLE` converts between the M3 traced reference and the C `void*`.

GC safety: During the callback, the closure `v` must not be collected or moved. Since `ForEach` holds a reference to `v` on the M3 stack, the GC will not collect it. CM3's conservative stack scanning ensures that the traced reference is seen even though we passed it as an untraced `ADDRESS`.

6.3 Closures with state

The predicate closure demonstrates captured state—the point coordinates `px`, `py` are fields of the closure object:

```
TYPE ContainsPred = PredicateClosure OBJECT
  px, py: LONGREAL;
```

```
OVERRIDES
  test := ContainsTest;
END;

(* Create and use: *)
VAR pred := NEW(ContainsPred, px := 1.0d0, py := 0.0d0);
    hits := ToyShapes.Filter(sl, pred);
```

The state travels through the C bridge via the `void*` context without the C code knowing anything about it.

7 Exception Handling

C++ exceptions cannot propagate through M3 stack frames. The C bridge catches all exceptions and converts them to error indicators:

```
toy_shape_t toy_circle_new(double cx, double cy, double r) {
    try {
        return new Circle(cx, cy, r);
    } catch (const std::exception &e) {
        set_error(e.what());
        return NULL;
    } catch (...) {
        set_error("unknown␣C++␣exception");
        return NULL;
    }
}
```

The M3 side checks for `NULL` and raises an M3 exception:

```
PROCEDURE WrapHandle(h: Handle): Shape RAISES {Error} =
  BEGIN
    IF h = NIL THEN RAISE Error(GetLastError()); END;
    ...
  END WrapHandle;
```

The error message is stored in a thread-local buffer on the C side and retrieved by `toy_last_error()`.

8 Linking

The C++ wrapper is compiled as a static library:

```
c++ -std=c++17 -O2 -fPIC -c shapes_c.cpp
ar rcs libtoyshapes.a shapes_c.o
```

The M3 `m3makefile` links it with `import_lib` and adds the C++ runtime via `SYSTEM_LIBS`:

```
import_lib("toyshapes", path() & "/../../toylib")
SYSTEM_LIBS{"CXX"} = ["-lc++"]
SYSTEM_LIBORDER += ["CXX"]
import_sys_lib("CXX")
```

On macOS, `-lc++` links the system `libc++.dylib`. On Linux, this would be `-lstdc++`.

9 Generalizing to Real Libraries

The patterns demonstrated here apply directly to binding any C++ library (Open3D, Eigen, CGAL, etc.):

1. Write a **C bridge** (`extern "C"` in a `.cpp` file) that exposes the needed functionality through opaque handles, flat arrays, and function-pointer callbacks.
2. Write an **M3 raw interface** (`UNSAFE INTERFACE`) that matches the C bridge exactly, with `EXTERNAL` declarations.
3. Write an **M3 idiomatic layer** that wraps handles in traced objects with GC cleaners, converts strings, and provides closure-based callbacks.
4. Link the C++ static library and `-lc++` into the M3 program.

The main cost is writing and maintaining the C bridge. For a large library like Open3D, this is significant work, but it only needs to cover the functions actually used.

9.1 Open questions

- **Shared libraries**: The prototype uses static linking. For large C++ libraries, dynamic linking (`libtoyshapes.dylib`) avoids code duplication but requires the library to be findable at runtime.
- **Thread safety**: The error buffer uses `thread_local`. If M3 and C++ both use threads, the interaction between the M3 GC (which may stop threads) and C++ thread primitives needs care.
- **Large data**: For million-element arrays (vertices, etc.), the copy-in/copy-out pattern works but doubles memory. A zero-copy approach could share a buffer allocated by one side and accessed by both, but requires careful lifetime management.
- **Subclassing from M3**: The current design does not allow M3 code to create new Shape subclasses that participate in C++ virtual dispatch. This would require a C++ proxy class that delegates virtual calls back to M3 methods—doable but complex.